TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

JBI010 - Assignment Grading Rubric

Total regular points: 100
Bonus points: 10

Task 1: Create Issues on GitLab (5 Points)

Criterion	Points	Description
Issue creation	**1.5**	At least one relevant GitLab issue is created for each required assignment task, and one for the bonus task if it is attempted.
Commit messages	**1.0**	Commits are made regularly and use meaningful messages.
Issue closure	**1.0**	Issues are closed when the corresponding work is completed.
Collaboration	**1.5**	Regular pull/push, conflict resolution, balanced partner contributions.

Partial Credit: *Missing some issues, vague commit messages, uneven partner contributions.*

- This assignment uses pytest. For **Tasks 2, 3, and 4**, comprehensive student-visible tests and a hidden staff test suite are used at grading time. The grader must run both. Points are deducted for any failing test or program crash.
- For **Tasks 2, 3, and 4,** students must also provide enough of their own pytests to thoroughly validate their implemented functionality. Deduct **1 to 5** points per task if no tests are provided or if the provided tests are insufficient.
- Documentation: Deduct up to **1 point per method** where docstrings are missing or incomplete.
- Type hints and naming: Deduct up to **1 point per method** where type hints are missing or unclear, or variable names are poorly chosen.
- For partial or inconsistent documentation or typing, apply proportional deductions based on the extent of the omission.
- **Use of libraries**: For **Tasks 2, 3, and 4,** using unauthorized third-party libraries not specified in the assignment may incur deductions of 1 to 5 points per task, depending on the severity and importance of the library used. Students may only use the libraries imported in the provided skeletons.
- **File structure:** The solution must be implemented in the provided files without renaming, moving, or restructuring them. The project uses a flat file structure with flat imports, such as `from track import Track`, and the sample tests are located in the tests folder. Solutions that reorganize the files into packages or change the expected layout may not be detected correctly by the automated grading system and may lose marks.

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Task 2: Define the Core Data Model (35 Points)

Part A: Implement the Track class (15 Points)

Criteria	Points	Description
`__init__`	4	Correctly initializes all 14 attributes with appropriate types, in the order listed in the Dataset section.
`__str__`	4	Returns a string exactly matching the required format (TrackID, Title, Artist, Genre) while reflecting the stored values.
`artist_names`	4	Splits the artists field on ';' and strips whitespace; a single artist returns a one-element list.
`audio_features`	3	Returns a dictionary of the six audio features (`danceability`, `energy`, `valence`, `tempo`, `loudness`, `acousticness`) with correct values.

Part B: Implement the MusicLibrary class (20 Points)

Criteria	Points	Description
`__init__`	2	The tracks attribute is initialized as an empty list named **`self.tracks`**.
`load_from_csv`	2	**Reading and loading:** Reads the CSV file and creates Track objects (e.g. via `csv.DictReader`).
	2	**Header normalization:** Converts column headers to lowercase and replaces spaces or hyphens in multi-word headers with underscores (for example, `Track Name` and `Track-Name` become `track_name`).
	2	**Categorical value normalization:** Normalizes valid categorical values to title case (for example, `pop` becomes `Pop`, and `hip-hop` becomes `Hip-Hop`).
	3	**Categorical validation:** Validates `genre`, `mood`, and `duration_category` against their allowed values. Checks 12 specified genres, `Sad`, `Neutral`, or `Happy` for `mood`, and `Short`, `Medium`, or `Long` for `duration_category`.
	2	**Numeric validation:** `popularity` is an integer in [0, 100]; `danceability`, `energy`, `valence`, and `acousticness` are floats in [0.0, 1.0]; `tempo` is a float greater than 0; `duration_ms` is an integer greater than 0; and `loudness` is accepted as any float.
	3	**Invalid, incomplete, and duplicate rows:** Does not load rows with invalid or missing fields, invalid categories, or out-of-range numbers. If a `track_id` occurs more than once, all rows with that identifier are excluded.
	2	**Robustness:** Does not crash on corrupted or malformed input; exceptions during validation are handled, and short or otherwise invalid rows are ignored.
`__len__`	1	Returns the number of stored tracks so that `len()` works on a `MusicLibrary`.

Part B: Implement the MusicLibrary class (20 Points)		
`get_track`	1	Returns the `Track` object with the matching `track_id`, or None if it is not present.

Task 3: Track Similarity and Recommendation (30 Points)

Part A: Feature Normalization (13 Points)		
Criteria	**Points**	**Description**
`__init__`	2	Correctly initializes `MusicAnalyzer` with the provided `MusicLibrary` object and stores it.
`feature_ranges`	4	Returns a dictionary mapping each of the six audio features to its `(minimum, maximum)` across all tracks. Returns an empty dictionary if the library contains no tracks.
`normalized_features`	4	**Min-max scaling:** Rescales each feature to [0.0, 1.0] using `feature_ranges`; reuses `get_track`, `audio_features`, and `feature_ranges`.
	3	**Edge case and error:** Maps a feature to 0.0 when its minimum equals its maximum and raises `ValueError` when the `track_id` is not present.

Part B: Distance and Recommendation (17 Points)		
Criteria	**Points**	**Description**
`distance`	4	**Correct value:** Computes the Euclidean distance between the two tracks' normalized feature vectors and reuses `normalized_features`.
	1	**Error handling:** Raises `ValueError` if either `track_id` is not present.
`recommend`	4	**Nearest tracks, self-exclusion:** Returns the `k` most similar tracks, excludes the query track itself.
	3	**Ordering and rounding:** Sorts recommendations by ascending unrounded distance and rounds each returned distance to four decimal places.
	2	**Deterministic tie-break:** Orders equal-distance tracks by `track_id` in ascending order; no randomness is used.
	1	**Handling of k:** Returns an empty list when `k` is 0. If fewer than `k` other tracks exist, returns all of them.
	1	**Error handling:** Raises `ValueError` if the `track_id` is not present or if `k` is negative. These checks run before **k** is used, so an invalid `track_id` raises even when **k** is 0.
	1	**Reuse of distance:** `recommend` is built on top of the `distance` method for modularity and reuse.

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Task 4: Playlist Generation and Analysis (25 Points)

Part A: Generate the Playlist (9 Points)

Criteria	Points	Description
`__init__`	1	Stores the ordered list of `Track` objects.
`generate_playlist`	4	**Playlist construction:** Starts from the seed and repeatedly appends the nearest unused track using the analyzer's `distance` method; the seed is the first track.
	2	**Determinism and tie-break:** Breaks ties using the smaller `track_id`; no randomness is used.
	2	**Edge cases and errors:** Raises `ValueError` if the seed is not found or `length` is less than 1. Includes all available tracks when `length` exceeds the library size.

Part B: Analyze the Playlist (16 Points)

Criteria	Points	Description
`total_duration`	2	Returns the sum of `duration_ms` over all tracks in the playlist. Returns 0 for an empty playlist.
`feature_averages`	3	Returns the mean of each of the six audio features, rounded to three decimals; reuses `audio_features`. Returns an empty dictionary for an empty playlist.
`popularity_statistics`	4	Returns a dictionary with keys "mean", "median", "std_dev"; mean and std_dev rounded to one decimal place. Returns an empty dictionary for an empty playlist.
`_median`	2	Returns a float in both cases: the middle value for an odd count, and the average of the two middle values rounded to one decimal place for an even count.
`dominant_mood`	2	Returns the most frequent mood; ties resolved to the mood appearing first in playlist order. Returns None for an empty playlist.
`artist_variety`	2	Returns the number of distinct individual artists (reuses `artist_names`, so collaborations are split and repeats counted once). Returns 0 for an empty playlist.
`_std_dev`	1	Correctly uses the provided sample standard-deviation helper.

Task 5: Interactive Visualization with Dash (Bonus: 10 Points)

Part A: Recommendation Panel (5 Points)		
Criteria	**Points**	**Description**
Seed and **k** controls	**1.5**	A way for the user to select a seed track (a dropdown of track names or a search by `track_id`) and that allows the user to choose a positive integer value of **k**.
Recommendation table	**2**	A table listing the results returned by the `recommend` method for the selected seed, with columns such as track name, artist(s), genre, and distance.
Dynamic update	**1.5**	The table updates dynamically whenever seed or **k** changes.

Part B: Generated-Playlist (5 Points)		
Criteria	**Points**	**Description**
Playlist generation	**1**	Generates a playlist for the selected seed using `Playlist.generate_playlist`.
Playlist table	**1.5**	Lists the generated playlist's tracks in order, with columns such as track name, artist(s), genre, mood, and duration.
Summary content	**1.5**	Displays the playlist's `total_duration`, `dominant_mood`, `artist_variety`, and normalized `feature_averages` chart.
Automatic update	**1**	The table, summary, and chart update automatically based on the user's selection.

Note: *All statistics and recommendations must be computed with the methods from Tasks 2–4 (not recomputed in the dashboard). Dash is not covered in the course and requires independent research. No credit for the bonus if the dashboard does not build on the previous tasks.*

Task 6: Reflection (5 Points)

Criteria	Points	Description
Issue description	**1**	Clearly describes two distinct bugs or problems encountered.
Diagnosis description	**2**	Explains how each issue was investigated and identified (root-cause analysis).
Resolution description	**2**	Details how each issue was fixed and the resulting outcome.

Note: *Limit to approximately one page (400–500 words). Evaluated subjectively to assess technical understanding and debugging approach. The Use of AI statement must be included as a separate section. Deduct 1 point if it is missing.*